

関連性の新たな役割：能動的検索におけるコーパスとの相互作用の 指針

Li Jiangnan^{1,*}, Li Yuqing^{2,*}, Yu Mo^{1,†}, Zhang Jinchao¹, Zhou Jie^{1,†}

¹ テンセント ² IIE-CAS

{jiangnanli,moyumyu}@tencent.com, liyuqing@iie.ac.cn

 github.com/LeqNaN/RARG

要旨

関連性とは、文書や抜粋に有用な証拠が含まれているかどうかを、クエリに基づいて推定したものである。既存のリトリブエージェントは関連性を利用して上位k件のコンテンツを選択しているが、文書関連性だけでは、複雑な質問に必要な証拠を特定・構成・検証することはできない。ダイレクト・コーパス・インタラクション (DCI) は、grepスタイルの探索を通じてこのようなきめ細かな操作を可能にするが、その関連性を考慮しない検索では、有用な手がかりの発見が遅れ、収束が遅れる可能性がある。最近の進歩では、関連性を利用してコーパスをインタラクションのための作業空間に絞り込んでいる。しかし、インタラクションが開始されても、関連性は依然として、grep検索でどの文書を最初に検索すべきかを直接的に示唆したり、広範な一致結果の中から有益な抜粋を区別してLLMに優先的に提示させたりすることはできない。我々は、関連性をコーパス相互作用の実行の事前条件に変換する「**関連性認識型 RipGrep 検索エージェント (RARG)**」を提案する。RARGは、粗いレベルから細かいレベルへの関連性ガイダンスを提供する。具体的には、順次RipGrep探索のために文書を並べ替えて全体として関連性の高い手がかりを早期に明らかにし、クエリに関連する段落を用いて有望なエントリポイントを初期化し、grepの一致結果を再ランク付けして、文書レベルのランキングでは見落とされがちな有益な抜粋を浮き彫りにする。困難なブラウズ型質問応答や推論を多用する検索において、RARGは検索ベースのエージェントや直接対話型エージェントと比較して、精度と効率のバランスを向上させます。これらの結果は、関連性を考慮した対話により、より迅速かつ信頼性の高い検索の収束が可能になることを示しています。

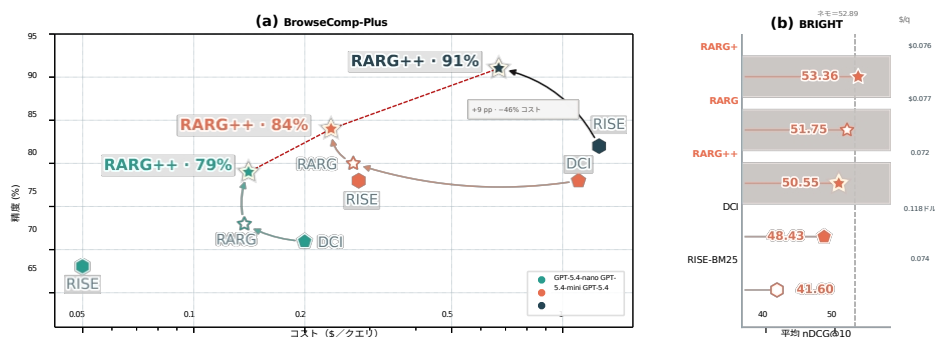


図1：BrowseComp-PlusおよびBRIGHTにおける精度／nDCG@10とインタラクションコスト（平均ツール呼び出し回数）の関係。RARGは、関連性をrg探索に対する実行の事前条件とすることで、検索ベースおよび直接インタラクション型エージェントに比べて、精度と効率のフロンティアを押し広げている。

* 同等の貢献；† 責任著者 プレプリント。

1 はじめに

関連性は、長きにわたり情報検索の組織化原理として機能してきた。本論文では、*関連性*とは、文書、文章、または一致した抜粋が、現在の情報ニーズに役立つ証拠を含んでいる可能性を、クエリに応じて推定することを意味する。RAG [7, 11, 14, 23] で広く採用されている現代の密な検索アルゴリズムは、この推定値を埋め込みの類似度 [10, 27, 32] によって具体化しているのに対し、疎な検索アルゴリズムは語彙的マッチング信号 [21] を用いる。いずれの場合も、関連性は有用ではあるが不完全な事前情報である。つまり、関連性は証拠がどこに見つかる可能性があるかを示すものであり、その証拠が十分であるか、正しく特定されているか、あるいは他の手がかりと適切に組み合わせられているかについては示さない。

検索エージェントは従来、この事前情報を用いてコーパスをランク付けし、上位 k 件の文書またはスニペットを言語モデルに提示している [9, 12, 13, 24, 28, 33]。このインターフェースはスケーラブルであるが、文書の関連性と証拠の有用性を混同してしまう。関連性の高い文書であっても、スニペットの切り詰めによって省略されたわずかな範囲に決定的な事実が隠されている可能性がある [17]。一方、マルチホップ検索では、その有用性が中間エンティティが発見されて初めて明らかになる場合もある [26]。クエリと文書の独立したランキングだけでは、そのような証拠を特定、関連付け、比較、または検証することはできない。したがって、先行研究で指摘されているように [16]、エージェントは適切な文書を検索できたにもかかわらず、答えを導き出せない場合がある。関連性はコーパス内の有望な領域を特定できるが、上位 k 件のランキング表示では、それらの領域内および領域間で証拠を活用するための支援は限定的である。

ダイレクト・コーパス・インタラクション (DCI) は、エージェントがパターンマッチングやローカルファイルの読み取りといった汎用的な端末ツールを使用して生ドキュメントを検索できるようにすることで、このインターフェース上のボトルネックに対処する [16]。きめ細かなインタラクションにより、エージェントは語彙的制約を組み合わせたり、新たに発見されたエンティティを追跡したり、一致箇所を取り巻く正確な文脈を精査したりすることができる。しかし、DCIではコーパスレベルの関連性ガイダンスがほぼ排除される。パターンマッチングコマンドは、すべての箇所が同等に有望であるかのように文書をスキャンするため、有用な手がかりが遅れて現れたり、出力が切り詰められた際に失われたりする可能性がある。その結果、エージェントは有用なアンカーに到達する前に、実りのない検索角度を探索するために多くの非生産的なインタラクションを費やすことになる。こうした反復的な相互作用が、無制限の検索を検証済みの証拠へとどれほど迅速かつ確実に絞り込むかを、我々は「*検索収束*」と呼ぶ。有望な箇所を優先順位付けするメカニズムがなければ、コーパスが大きくなるにつれてDCIの収束性は低下し、ツールの使用頻度が急速に増加し、精度が低下することになる [16]。

Pi-Serini, RISE, DR-DCI を含む最近の進歩は、インタラクションの前または最中に、スパースまたはデンシな検索を用いて境界のある候補空間を構築または拡張することで、このスケーラビリティ問題に対処している [8, 18, 35]。インターフェースには違いがあるものの、これらには共通の戦略がある。まずコーパスを関連性の高い文書の集合に絞り込み、次にエージェントがこの集合を、選択的な読み取りや `grep` スタイルの操作を通じて探索できるようにする。これにより、コーパス全体の繰り返しスキャンを回避しつつ、従来のトップ k スニペットインターフェースよりも豊かなインタラクションを維持できる。しかし、関連性は依然として主に空間を構築するためのメカニズムであり、その空間内でのインタラクションに対するきめ細かな実行シグナルとはなっていない。関連性は、パターンマッチング操作の探索順序や、出力の切り捨ての下でどの局所的な一致が可視のまま残るかを直接決定するものではない。

我々は、検索の収束を加速・安定化させるためには、関連性が単にコーパスの入力を選択するだけでなく、コーパスとのインタラクションそのものを導くべきであると主張する。これには、互いに補完し合う2つの解像度における関連性が必要となる。*グローバルな関連性*は、パターンマッチング操作が最初に検索する場所を決定し、関連性の低い文書よりも有望な文書が先に検出されるようにすべきである。*局所的な関連性*は、生の出力がエージェントの観察予算を超えた際に、どの一致した抜粋が表示されるかを決定すべきである。この見解は、DCIの高解像度で構成可能なインターフェースを維持しつつ、検索スコアを厳格な証拠のボトルネックではなく、実行の事前情報として活用するものである。

この原則に基づき、我々は「**関連性認識型RipGrep検索エージェント (RARG)**」を導入する。² RARGは、エージェントが生成したクエリに従って埋め込みリトリーバーを用いて文書をランク付けし、その後、`rg`にその順序で文書を順次探索させる。これにより、関連性の高い文書からの一致が早期に抽出され、文書レベルの関連性がトップ k コンテンツの選択ではなく、検索順序へと変換される。さらに、2つの拡張について検討する。RARG+は、エージェントにクエリに関連する少数の段落をエントリポイント [15] として与え、そこから最初の精密な検索を構築できるようにする。RARG++は、グローバルな検索目的とローカルなパターンマッチングの焦点を組み合わせることで、より広範な`rg`の一致候補を再ランク付けし、これにより、ランクが低い文書に含まれる証拠を含め、局所的に有益な抜粋が、エージェントの限られた観察リソースを競い合えるようにする。これら3つのレベルが相まって、どこを

² 本論文全体を通じて、`grep`は一般的な`grep`スタイルのパターンマッチング操作を指し、`rg`は当実装で使用される具体的な`ripgrep`コマンドを指す。

相互作用がどこで始まるか、どの文書を最初に探索するか、そしてどの局所的な観測がモデルに到達するかを決定し、これにより rg が有用な証拠に早期に到達し、非生産的な相互作用を最小限に抑えて収束できるよう支援する。

我々は、2つの補完的な設定において本手法を評価した。1つは、RISE [1, 35] に準拠した100クエリのBrowseComp-Plus設定における難易度の高い質問応答であり、これには文書数を10万件から100万件へと制御的に拡張したケースが含まれる。もう1つは、4つのBRIGHTドメイン [25] における推論を多用する検索である。図1に示すように、10万文書のBrowseComp-Plusコーパスにおいて、GPT-5.4-miniを使用した場合、RARG++は84%の精度を達成した。これに対し、RISEとDCIはいずれも78%の精度にとどまり、ツール呼び出し回数の平均は、それぞれ28.7回および99.1回であったのに対し、RARG++は23.9回であった。GPT-5.4を使用した場合、RARG++は91%の精度を達成し、RISEを9ポイント上回った。コーパスを100万文書に拡張した後も、RARG++は79%の精度を維持したのに対し、RISEは69%にとどまった。BRIGHTでは、RARG++は平均nDCG@10で53.36を達成し、DCI、RISE、および検索特化型のNeMoエージェントを上回った。精度とツール効率の向上は、関連性を意識した対話により、検索の収束がより迅速かつ確実になるだけでなく、推論を要する証拠の特定も改善されることを示している。

本研究の貢献は以下の3点である：

- 我々は、エージェント型検索において関連性が果たすべき、これまで見落とされていた役割を特定した。すなわち、検索されたコンテンツの選択やワークスペースの管理にとどまらず、関連性はコーパスとの相互作用の実行と観察を直接的に導くことができる。
- 我々は、コーパスの探索中に文書の順位を維持する関連性を考慮した rg 検索エージェント「RARG」を提案し、より迅速かつ信頼性の高い検索収束を実現するために、粗いレベルから細かいレベルへのガイダンスを提供する「エントリポイント初期化」と「マッチレベル再ランク付け」を導入した。
- 我々は、困難なQA、コーパスのスケールアップ、および推論を多用する検索において、検索のみを行うエージェント、制限のないDCI、および検索によって構築された相互作用空間と比較して、より優れた精度と効率のトレードオフを実証した。

2 関連研究

2.1 文書検索からエージェント型検索へ

関連性の推定は、現代の情報検索における中心的な組織原理である。BM25のようなスパースな手法は、語彙統計を用いて文書をランク付けする [21] 一方、デンセンな検索エンジンは、意味的マッチングのためにクエリと文書を共有表現空間にエンコードする [10]。

検索拡張生成 (Retrieval-augmented generation) は、このような検索手法を用いて言語モデルを外部知識に結びつけるものであり、長い入力 [3, 5, 29]、多ショットデモンストレーション [4]、コンテキスト圧縮 [2]、およびグローバルに分散した証拠 [11] を扱う設定で研究されてきた。しかし、これらのアプローチは依然として、推論の前に組み立てられたコンテキストに依存しており、そのコンテキストは下流の証拠ニーズを予測しなければならない。一方で、ランク付けされたコンテキストが長くても、有用性の低い情報の中に埋もれた決定的な証拠をモデルが確実に活用できるとは限らない [17]。

「エージェント型検索」は、モデルが複数のステップにわたって検索を制御できるようにすることで、このワンショットという仮定を緩和する。推論と検索アクションを交互に行うことで、中間的な推論結果がその後の情報ニーズを形作ることができるようにする [26, 28, 30, 31, 34]。しかし、そのようなシステムの多くは依然として、従来型のリトリバーを主要なコーパスインターフェースとして扱い、トップkの結果を返すだけであり、エージェントは検索完了後にのみそれらについて推論を行うことができる。この設計では、文書レベルの関連性と証拠の有用性が混同される可能性がある。決定的な手がかりは、文章の選択過程で省略された短い範囲に存在しているかもしれない一方で、文書の価値は、中間的なエンティティが発見されて初めて明らかになる場合もあるからだ。こうした制限を背景に、スケラブルなコーパスレベルの優先順位付けと、よりきめ細かな証拠の特定、構成、検証を組み合わせたコーパスインターフェースが求められている。

2.2 エージェント指向検索のためのコーパスインターフェース

検索品質に加え、重要な設計軸となるのが、エージェントがコーパスにアクセスするためのインターフェースの解像度である。Pi-Serini [8] のような強力な語彙パイプラインを含むリトリバーを介したシステムは、ランク付けされた文書ビューを提示するため、エージェントを事前に選択された証拠にほぼ限定してしまう。ダイレクト・コーパス・インタラクション (DCI) は、この制約を高解像度のインターフェースに置き換え、生のコーパスをファイルとして公開し、エージェントがパターンマッチング、ローカル読み取り、シェルコマンドを通じて操作できるようにすることで、厳密な語彙制約、エンティティ追跡、およびファイル間の合成をサポートする [16]。GrepSeekはさらに、このようなインタラクション方針がコンパクトな

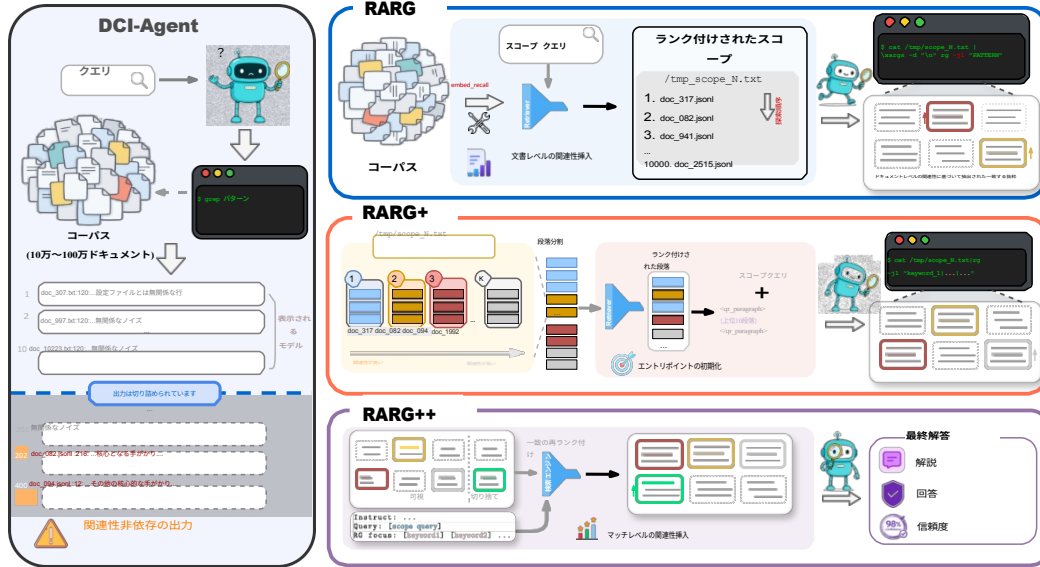


図2：RARGの概要。*embed_recall*はコーパスをスコープファイルにリンク付けし、*rg*はリンク順にスキャンして関連性の高い文書を最初に表示する（RARG）。RARG+はクエリに関連する段落をエントリポイントとして追加し、RARG++は*rg*の一致結果を再ランク付けして、ランクの低い文書からの有益な抜粋も可視化されるようにする。文書レベルの関連性は、*rg*が最初に検索する場所を決定し、一致レベルの関連性は、どのマッチがモデルに到達するかを決定する。

検証済みの軌跡と強化学習を用いたエージェント [22]。しかし、この柔軟性には、コーパスレベルでの優先順位付け機能が限定されているという課題が伴う。制限のない操作では、すべてのファイルが同等に有望であるかのようにコーパスを横断してしまう可能性があり、その結果、妨害要素の数に比例してコストが増大してしまう。

RISE [35] および DR-DCI [18] は、直接的な相互作用に制限を設けることで、このスケーラビリティのジレンマに対処している。RISEは、検索を「インタラクション空間の構築」として再定義し、BM25を用いてシェル形式のナビゲーション用に前処理された部分集合を選択する。一方、DR-DCIは、検索をエージェントが呼び出せるアクションとして扱い、永続的なワークスペースを段階的に拡張する。いずれのアプローチにおいても、関連性は主に検索可能な空間の構築や拡張に用いられるが、生の出力が観測予算を超えた際に、局所的な探索順序を決定したり、一致項目の優先順位を付けたりするために明示的に伝播されることはない。

これにより、コーパスレベルの関連性を推定することと、インタラクション中にそれを利用することとの間にギャップが生じている。我々のアプローチでは、関連性を検索可能なコーパスに対する一回限りのフィルタとして扱うのではなく、実行の事前条件として直接的なコーパスインタラクションに持ち込み、限られた観測予算の下で、検索の開始位置、文書の探索順序、およびどの局所的な一致がモデルに到達するかを導く。

3 方法

*grep*は関連性ではなくキーワードやパターンに一致させるため、そのヒット結果は質問と無関係である可能性があり、エージェントに無駄な検索ステップを強いることになる。そこで、図2に示すように、*grep*をより適切かつ迅速な探索へと導く「関連性を考慮したRipGrep（RARG）」エージェントを導入する。

3.1 DCI ベースのエージェント構造

RARGは、以下の2つのツールのみを提供する検索エージェント「DCI-Agent-Lite」を基盤として完全に構築されています：

Bash(`command`) および *Read*(`doc_path` , `offset` , `limit`)。 *Bash*は主に*grep*コマンドを実行してコーパス全体からローカルな一致を検索し、*Read*は一致した行が `オフセット` を `offset+limit` ドキュメントの返します。

ツールの引数はLLMによって自由に生成されるため、*Bash*には次のようなコマンドが送信される可能性もあります。

{ls, find, python, cat, ...}のようなコマンドを受け取る可能性があります。コーパスは非常に大規模になる可能性があるため、

実行すると長時間停止してしまう可能性があるため、コーパス全体を対象とする `ls/find` の実行は禁止する。

エージェントは、回答に十分な確信が持てるまでツール呼び出しを継続しますが、無制限の対話によりコンテキストがすぐにオーバーフローしてしまいます。DCI-Agent-Lite は、4 段階のコンテキスト管理によってこの問題を軽減しています。ここでは、デフォルトのレベル 3 の圧縮を採用しています。(1) 個々のツール結果を C 文字に切り詰める。(2) コンテキストの長さが閾値を超えた場合、直近の T ターン分のツール結果を保持し、それより古いものは「[cleared]」というプレースホルダーに置き換える。

3.2 文書レベルの関連性挿入

前述の通り、`grep` が手がかりをより迅速かつ確実に特定するためには、関連性が依然として不可欠です。中心となる課題は、どのようにして `grep` に関連性を組み込むかということです。質問に対する関連性に基づいて文書をランク付けする最も一般的な方法である「検索機能の組み込み」は、上位にランク付けされた文書を LLM に直接供給する場合、単純な `grep` よりも性能が劣ることが知られています[16]。そこで我々は、リトリバルを証拠チャネルそのものとしてではなく、`grep`（「キャリアー」）に関連性の指針を提供する「サポート」として扱うことにした。

まず、新しいツールである `embed_recall` (`scope_query`) を介して、`grep` に文書レベルの関連性を注入し、

を介して、`grep` に文書レベルの関連性を注入します。このツールは、LLM が生成したクエリを受け取ります。文書の内容を返すのではなく、ランク付けされた文書のパスを一時ファイル `\tmp\scope_{N}.txt` に書き出し、LLM には「`\tmp\scope_{N}.txt` → クエリ」というマッピングのみを返します。非常に大規模なコーパスの場合、処理コストを抑えるため、また関連性の高い文書はほぼ常に上位にランクされるため、すべてのパスを記録する必要はありません。そこで、スコープを 10,000 パスに制限しています。それでも非常に高いエビデンスリコール率が得られます——これが「scope」という名称の由来です。その後、LLM に対し、スコープファイルの内容を表示し、**パスの順序に沿って検索するよう**指示します。これにより、意味的に類似度の高いドキュメントの一致がより早い段階で抽出されます。具体的には、LLM に対して以下のコマンドを使用するよう指示します：

```
cat /tmp/scope_{N}.txt | xargs -d '\n' rg [OPTIONS] "PATTERN"
```

これにより、スコープ内のすべてのドキュメントパスが `rg` ブロックに追加され、検索対象がそれらのドキュメントに限定されます。しかし、`rg` はファイルを並列（マルチスレッド）で読み込み、スレッドが処理を完了した順に一致結果を出力するため、私たちが指定したパスの順序が崩れてしまいます。そのため、`rg` の呼び出しを正規表現でマッチさせ、`rg -j1 [OPTIONS]` を挿入して、順次かつシングルスレッドでのスキャンを強制します（`rg -j1 [OPTIONS]`）。

LLM には、各エピソードの開始時に、元の質問を引数として `'embed_recall'` を呼び出し、`scope_query` (内部の挿入) を呼び出し、その後、結果のスコープ内で `rg-searching` (内部の挿入) を行い、その後 `rg-search` を実行するよう指示されます。また、`embed_recall` を呼び出して新しいスコープを形成することも可能です。圧縮処理中、「`\tmp\scope_{N}.txt` → スコープクエリ」のマッピングはクリアされるのではなく保持されるため、LLM は常に利用可能なスコープを把握しています。この手法を RARG と呼びます。付録 B のプロンプトおよびツールの定義を参照してください。

3.3 エントリポイントの初期化

エージェントは質問のみから開始し、`'embed_recall'` はそれを充実させるためのコンテンツを追加しないため、`rg` は収束に適したエントリポイントを見つけるまでに数ステップを無駄にする可能性があります。追加の LLM ターンを行わずにそのようなエントリポイントを提供するため、上位にランク付けされたスコープ文書からクエリに関連する数段落を `'embed_recall'` の出力に追加します。

具体的には、スコープの上位 X 件の文書を 400～1000 文字の段落に分割し、埋め込みモデルでエンコードした後、スコープのクエリに対してスコア付けを行い、上位 10 件を保持します。これらの段落は `<qr_paragraph></qr_paragraph>` で囲まれ、スコープマッピングの後に追加されます。これらは回答そのものではなく、LLM の開始点として機能します。これらは圧縮時にクリアされます。この初期化処理を施した RARG を、RARG+ と表記します。

3.4 マッチレベルでの関連性挿入

収束をさらに加速させるため、各検索ステップでは、個々のマッチした抜粋と検索目標との間のきめ細かい関連性も考慮する必要があります。3.1 で述べたように、`rg` の出力は切り詰められ、特に RARG の上位ランクの文書内において、一部の一致のみ

が保持されます。文書レベルの関連性のみを考慮すると、有益な抜粋が周囲の文書

によって希薄化され、その文書が低い順位となり、抜粋が見えなくなってしまう可能性がある。そこで、我々は埋め込みモデルを用いて、より広範な一致候補のプールを再ランク付けし、上位のもののみを提示する。

ステップごとの一致結果の再ランク付けにおける課題は、埋め込みモデルが、グローバルな検索目標とローカルな rg の意図の両方を捉えた専用のクエリを必要とする点である。我々は、(1) 構築されたクエリと (2) 生成型クエリという 2 つの選択肢を検討する。

構造上、`embed_recall` の呼び出しはスコープとそのクエリを生成し、rg はそれを数ステップにわたって探索します。スコープクエリはグローバルな目標を捉え、rg のキーワード／パターンはローカルな意図を捉えます。したがって、rg が `\tmpscope_{N}.txt` をキャットした際、そのパターンをルールに従ってキーワードシーケンスに変換し、次のようにスコープクエリと連結します：

```
Instruct : ... Query : [  
スコープクエリ]  
RGの焦点 : [ キーワード1 ] [ キーワード2 ] ...
```

この構築されたクエリは、最大 M 件の一致を再ランク付けし、上位 m 件 ($m < M$) を選択することで、より多くの候補手がかりが LLM に到達する機会を与えます。一致件数が m 未満の場合は、再ランク付けはスキップされます。

生成型クエリについては、`Bash` を `RerankAwareBash( command , rerank_query =None )` に置き換えます。

ここで、LLM は `rerank_query` を生成するよう指示されており、そうでない場合は `None` のままにします。

LLM には、`rerank_query` このクエリは、最終目標と局所的な検索

意図を反映したクエリを生成するよう指示される。しかし、我々の実験では、この生成型バリエーションは性能を低下させた。マッチレベルでの再ランク付けを行う RARG を RARG++ と表記し、デフォルトでは構築されたクエリを使用する。

4 実験

4.1 実験設定

ベンチマーク。我々は、2 つのシナリオ（難易度の高いブラウズ型質問応答と推論を多用する検索）において、本手法の有効性と効率性を評価する。QA については、RISE に準拠し、固定の 10 万文書コーパス上で困難なブラウズ型質問に答える BrowseComp-Plus [1] (BC+) を使用する。コストの都合上、RISE は 100 クエリのサンプルで評価を行っている。より広範な検索空間を調査するため、FineWeb-Edu からサンプリングした 90 万件の長いノイズを含む文書でコーパスをさらに拡張した。これは、文書が長くなるほど、ノイズを含むコンテンツとの偶発的な一致が生じる機会が増え、検索が困難になるためである。精度は、DCI の「LLM-as-judge (GPT-5.1)」プロンプトを用いて算出する。検索に関しては、DCI に準拠し、BRIGHT [25] の 4 つのサブセット（生物学、地球科学、経済学、ロボット工学）を使用する。BRIGHT は、表面的な意味的マッチングではなく、深い推論を必要とするため、従来の IR ベンチマークよりも難易度が高い。これら 4 つのサブセットには、57K/121K/50K/62K の文書からなるコーパスに対して、それぞれ 103/116/103/101 の質問が含まれており、nDCG@10 の値を報告する。

実装の詳細。すべての実験を、GPT-5.4-mini（中程度の思考負荷）、GPT-5.4-nano（高程度の思考負荷）[19]、および GPT-5.4（中程度の思考負荷）[20] を用いて実行した。第 3 節で説明したように、RARG は DCI-Agent-Lite ハーネスを基盤として構築されており、若干の適応が施されている。rg については、返される一致結果の数を BC+ では 30 件、BRIGHT では 60 件に制限し、各一致結果を BC+ では 1,000 文字、BRIGHT では 500 文字に切り詰めています。その結果、出力は最大で $30,000 + 30 \cdot (60 \times \text{path_name_length})$ 文字となり、DCI の 50 KB をはるかに下回ります。DCI の圧縮機能は直近 12 ターン分のツール結果を保持しますが、GPT は 1 ターンあたり複数のツールを呼び出すことが多いため、これでは結果が過剰に保持されてしまいます。そこで、代わりに直近 40 件のツール結果を保持し、キャッシュをより有効に活用するために圧縮のしきい値を 230K に引き上げました。BC+ については、ドキュメントのランキング、初期化、およびマッチの再ランキングに Qwen3-Embedding-4B [32] (Q3E) を使用する。BRIGHT については、文書ランキングと初期化に llama-nv-embed-reasoning-3b³ (NV) を、マッチの再ランキングに Q3E を使用する。NV は、BRIGHT で最も優れたエージェントである NeMo [6] が使用する推論集約型の埋め込みモデルであり、我々は NeMo の設定に従う。ただし、NV は短いマッチのランキングにおいて性能が低いことが判明したため、再ランキングには Q3E に切り替えている。再ランク付けのプールは $M = 500$ 、 $m = 30/60$ に設定し、ターン数は 100 に制限した。RARG は Python で実装され、1 枚の H20 GPU を搭載した Linux 上でテストされた。

³ <https://huggingface.co/nvidia/llama-nv-embed-reasoning-3b>

表1：10万文書のコーパスを用いた100クエリ（RISEバージョン）のBrowseComp-Plusにおける主な結果。「Turns」および「Tools」は、エージェントの平均ターン数およびツール呼び出し回数を示す。「Search」、「Bash」、「Read」は、各ツールへの平均呼び出し回数を報告する。本手法において、「Search」の呼び出しは*embed_recall*である。最高のAccおよび最小のTurns/Toolsは太字で示されている。Search/Bashの一部には、エージェントの異なる挙動を示すために下線が引かれている。△は、結果がRISE [35]から引用されたことを示す；?は、RISEにおいてどの思考プロセスが活性化されたかについて言及されていないことを示す。NeMoの左側のツール呼び出しは、*Think* - 1.3/0.5（mini/nano）および*Answer* - 0.8/0.7である。

モデル	手法	Acc↑	ターン数	ツール	検索	バッシュ	読む
GPT-5.4 mini (中程度の労力)	RISE [△]	78	24.3	28.7	13.1	9.2	6.4
	RISE-BM25 [△]	77	23.0	29.6	<u>14.8</u>	<u>9.6</u>	5.2
	RISE-Q3-Emb-4B	69	28.9	35.9	22.1	9.1	4.7
	検索エージェント [△] 埋め込み	68	29.2	38.9	37.0	—	1.9
	エージェント	55	54.3	68.0	68.0	—	—
	Nemo Agent	64	25.3	30.3	28.2	—	—
	DCI [△]	78	48.8	99.1	—	<u>90.3</u>	8.8
	RARG	80	18.2	29.8	<u>1.2</u>	<u>27.2</u>	1.4
	RARG+	81	20.2	29.6	<u>1.6</u>	<u>26.6</u>	1.3
	RARG++	84	17.6	23.9	<u>1.5</u>	<u>21.1</u>	1.3
	RARG++ (生成型)	75	15.8	17.8	1.2	15.4	1.3
GPT-5.4-nano (高負荷)	上昇 [△]	68	23.4	28.7	10.4	8.4	4.6
	RISE-BM25 [△]	64	22.0	29.6	10.2	8.1	3.7
	エンベディング・エージェント	49	43.1	66.6	66.6	—	—
	NeMoエージェント	57	12.1	37.8	36.6	—	—
	DCI [△]	71	45.7	126.5	—	119.4	7.1
	RARG	73	39.9	41.4	<u>9.1</u>	29.5	2.8
	RARG+	74	40.2	39.8	<u>14.5</u>	22.4	2.9
	RARG++	79	36.0	36.1	<u>9.6</u>	23.4	3.1
GPT-5.4 (中程度の運動量)	RISE ^{?△}	82	32.20	34.30	<u>11.00</u>	<u>16.20</u>	7.10
	RARG++	91	13.59	25.43	<u>1.58</u>	<u>19.46</u>	4.39

比較対象のエージェント手法。以下のエージェントと比較する。DCI-Agent-Lite [16]は、BashとReadのみを備えた純粋なgrepエージェントである。RISE [35]は、BM25検索によって相互作用空間を構築し、検索された文書にナビゲーション構造を追加した後、エージェントがシェルツールを使用してその空間を探索する。RISE-BM25は、BM25によるインタラクション空間を維持しつつ、このナビゲーション処理を行わず、元の構造化されていない文書を使用するアブレーション実験である。DCI-Agent-LiteおよびRISEベースのベースラインについては、公式のRISE実装⁴を使用して実行した。さらに、BM25リトリバーをQ3Eに置き換えたRISE-Q3-Emb-4Bと、RISEとともにリリースされた従来のリトリブベースラインであるRetrieval-Agentについても評価を行った。Embedding Agentは、Q3Eベースのdense retrievalを通じて候補文書を検索するエージェントである。NeMo Agent [6]は、候補文書を反復的にリコールおよびランク付けして最終出力を形成する、検索指向のエージェントであり、その公式実装⁵を使用する。これらのエージェントの実装の詳細は付録Aに記載されている。

4.2 難易度の高いQAにおける結果

表1に示すように、RARGはすべてのバックボーンにおいて最高の精度を達成しつつ、インタラクションコストを大幅に削減している。RARG++は、GPT-5.4-mini/nanoで84%/79%、GPT-5.4で91%に達し、最有力なベースラインを6/8/9ポイント上回っているが、ツール呼び出し回数は23.9回／25.43回にとどまるのに対し、DCIは99.1回、RISEは28.7回／34.3回（GPT-5.4-nanoの設定を除く）である。これは、我々の中心的な主張を裏付けるものである。すなわち、関連性を無視するDCIの精度を、関連性を無視するのではなく、関連性に基づいて対話に誘導することで、はるかに低コストな検索へと変換できるのである。

Embedding AgentとNeMo Agentは、検索中心のインターフェースを分離している。NeMoを含めたのは、そのリコール・アンド・ランク・ハーネスが重複を排除し、より多くの異なる文書をリコールするためであり、これにより、Embedding Agentのこのようなカバレッジが強化されたバリエーションが、証拠をより適切にモデル化しているかどうかを検証できるからだ。しかし、両者ともBC+では依然として最も弱い結果を示している。これらは、探索の指針としてではなく、トップランクのコンテンツとしてのみ関連性を露呈しており、まさにLiら[16]が指摘したインターフェースのボトルネックそのものである。

下線部の「Search/Bash カウント」は、関連性がどのように利用されているかを明らかにしている。RISEは、検索を用いてコーパスを限定し、rg探索ステップを削減する作業空間を構築する。しかし、空間を構築するすべてのSearchは、上位文書のスニペット（最初の数百文字）も返すため、検索

⁴ <https://github.com/texttron/RISE>

⁵ <https://github.com/NVIDIA/NeMo-Retriever/tree/main/retrieval-bench>

表2：GPT-5.4-mini（中程度の推論能力）を用いて、10万件のドキュメントから100万件のドキュメントへと100クエリのBrowseComp-Plusをスケールアップした結果。

モデル	手法	精度 [†]	ターン数 [‡]	ツール [‡]	検索	Bash	読む
GPT-5.4 mini (中程度の運動量)	RISE-BM25	69	25.4	32.1 (+2.5)	20.3 (+5.5)	8.3 (-1.3)	3.5 (-1.7)
	RARG	78	19.3	31.8 (+2.0)	1.5 (+0.3)	28.7 (+1.5)	1.7 (+0.3)
	RARG+	78	21.9	30.4 (+0.8)	1.5 (-0.1)	27.5 (+0.9)	1.4 (+0.1)
	RARG++	79	17.8	24.7 (+0.8)	1.4 (-0.1)	22.0 (+0.9)	1.4 (+0.1)

同時に、モデルへの情報チャンネルとしても機能します。各Search呼び出しが直接コンテンツを供給するため、これによりSearch呼び出しの増加を促してしまうリスクがあります。RISEはBM25を前提に設計されているのに対し、我々はQ3Eを使用しているため、リトリバーの性能を制御するためにRISE-Q3-Emb-4Bも実行した。しかし、これは効果がなく、実際にはより多くのSearch呼び出しが発生した。これは、RISEがより強力な高密度リトリバーではなく、BM25インターフェースに適応したままであることを示している。RARGはこのバランスを逆転させる。nanoを除き、ドキュメントレベルの検索順序を決定するために

1.2〜1.6回のembed_recall呼び出しのみを行い、ドキュメントレベルの検索順序を確定させた後、探索の大部分をスコープ付きrgに委譲する。したがって、関連性はrgのトラバーサルに対する実行の事前条件として機能し、証拠チャンネルそのものとしては機能しない。nanoでは、このパターンはそれほど明確ではなく（embed_recallの値が高く、変動も大きい）、これは指示の順守が弱いことに起因すると考えられ、挙動分析で改めて検討する。それにもかかわらず、GPT-5.4-nanoを用いた我々の手法は依然として最高の性能を達成している。

RARG内では、各バリエーションが意図した「粗い段階から細かい段階へ」という進行をたどっている。RARGからRARG+、RARG++へと進むにつれ、miniの精度は80→81→84に上昇し、ツール数は29.8→29.6→23.9に減少しており、nanoでも同様の順序（73/74/79）が見られる。したがって、関連性の各レベルは精度を向上させ、収束を加速しており、これは我々の3つの貢献を直接反映している。生成型バリエーションはこの傾向を破っている：収束は最も速い（17.8ツール）が、精度は75%に低下する。我々はこれを学習と評価のギャップに起因すると考えている。モデルに明示的な再ランク付けクエリを出力させることは、学習時に用いられたBash/rgの挙動を乱すためである。収束は改善されるため、このギャップを埋めることは今後の研究における有望な方向性である。

4.3 コーパスの拡張

検索空間が拡大しても、関連性に基づく相互作用が有用であるかどうかを検証するため、BC+コーパスを10万文書から100万文書に拡張した。追加された90万文書はFineWeb-Eduの長い記事であり、したがって、偶発的な語彙の一致や無関係な文脈が発生する可能性が大幅に高まる。

表2に示すように、100万ドキュメントへの拡張によりすべての手法の精度が低下し、RISE-BM25は77%から69%へ、RARG/RARG+/RARG++は80/81/84から78/78/79へと低下した。これは、RISE [35] がスケールリング下で報告している非減少傾向とは矛盾する。この違いは設計に起因する可能性がある。我々が追加した90万件の文書はFineWeb-Eduの長い記事であり、これによって偶発的な語彙の一致が発生する可能性が大幅に高まり、各rgスキャンにはるかに多くの注意をそらす文脈が混入するからである。すべての検索エージェントで性能は低下するが、ツールの数がわずかにしか増えないため、コストはスケールリングの影響をあまり受けていないようだ。その理由として、検索エンジン（Q3E や BM25）への影響は比較的少ないものの、rg は注意をそらす要素の影響を免れていないことが考えられる。ファイルごとのノイズが増加すると、通常の局所的な語彙検索と同様に、一致の品質が低下する。こうした注意をそらされた局所的な一致は誤りを生じさせ、LLM の判断を誤らせる。この主張を裏付けるため、第 4.5 節で 1M コーパスにおける RARG の挙動を分析する。RARG++はコーパスの規模拡大に伴い性能が低下するものの、RISE-BM25に対して10ポイントのリードを維持している。コーパスに起因する干渉は、マッチレベルでの再ランク付けによって部分的にしか吸収されないため、ローカルマッチングのこの弱点をどのように軽減するかは、今後の研究課題となり得る。

4.4 理由重視型検索に関する結果

BRIGHTとBC+は、対照的な検索パターンを重視しています。ランキング形式の検索では、エージェントはまず関連性の高い文書をできるだけ多く抽出し、その後それらを正確に順位付けすることが求められます。これは「幅優先」のプロセスと見なすことができます。対照的に、QAはより「深さ優先」的です。つまり、正しい答えに到達するための、検証済みで簡潔な推論経路が1つあれば十分です。したがって、RARGが最適化を目指す高速収束はBRIGHTとの相性があまり良くないが、表3が示すように、RARG+は依然として最高の平均nDCG@10（53.36）を達成しており、検索に特化したNeMo（52.89）や他のすべてのベ

ースラインを上回っている。したがって、目的が単一の回答ではなくランク付けされたリストである場合でも、関連性を考慮した対話には依然としてメリットがある。

表3：BRIGHTにおける検索の有効性（NDCG@10で測定）。特に指定のない限り、各手法はGPT-5.4-mini（中程度の推論能力）を使用している。

手法	平均↑	Bio.	Earth	Eco.	ロボット	ツール	検索	Bash	読む	考える	回答
DCI	48.43	62.05	54.94	37.13	39.59	40.04	—	14.73	25.31	—	—
RISE-BM25	41.60	50.27	47.80	33.65	34.67	31.82	7.55	2.15	22.12	—	—
ネモ・エージェント	52.89	65.15	61.85	39.05	45.49	7.68	6.36	—	—	0.37	0.95
RARG	51.75	63.87	60.54	38.50	44.07	28.73	1.10	11.25	16.38	—	—
RARG+	53.36	66.70	62.16	37.23	47.34	27.55	1.23	9.87	16.45	—	—
RARG++	50.55	61.65	61.32	36.14	43.10	27.28	1.14	8.40	17.74	—	—

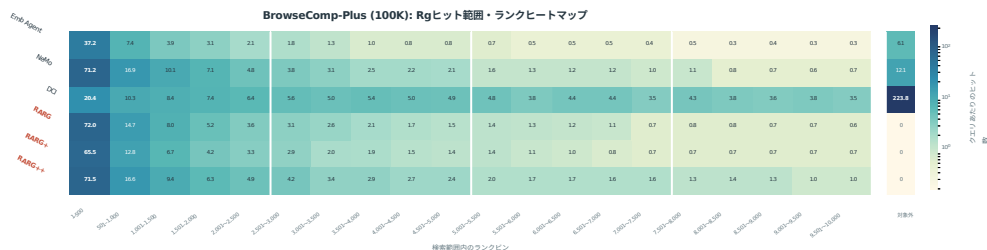


図3：BC+における、範囲内での関連文書ヒット数の推移。範囲外の手法ではRARGのscope_1.txtを使用している。埋め込みベースのヒットは上位に集中しているが数が少なく、DCIは平坦に関連性には依存せず、RARGは上位に偏った状態を維持する一方、マッチレベルでの再ランク付けは低順位の文書から証拠を回収する。

ここでのツール数は異なるインターフェースを反映しており、コストの比較対象ではない。NeMoはリコール・アンド・ランク型エージェントであり、1回のSearchにつき20件の全文文書を取得し、強力なLLMによる再ランク付けに依存している。また、BRIGHTの文書は（BC+とは異なり）比較的短いため、このループは非常にコンパクトである（7.68ツール）。一方、grep指向のエージェントは局所的な一致のみを観察し、周囲の文脈を把握するためにReadを実行する必要があるため、Readツール呼び出し数が多くなる。DCIについては、関連性を考慮しないrgは探索ステップ数が着実に増加するものの、関連性の低い情報しか提供しないため、パフォーマンスが低下する。RISEは依然としてSearchに依存しているが、BM25では明らかに理由重視の検索を処理できない。RARG内では、QAと比較して順位が逆転する：RARG+ > RARG > RARG++ (53.36/51.75/50.55)。適切な初期化が与えられたRARG+は、最高のパフォーマンスを達成する。しかし、Bash呼び出し回数からわかるように、RARG++のマッチレベルでの再ランク付けは、局所的に優れた抜粋に観察リソースを集中させ、比較的最も速く収束する。これはQAにとっては利点だが、タスクが広範なりコールを重視する場合は欠点となる。全体として、RARGは関連性を活用してBRIGHTを上回るが、最適な粒度は、タスクが迅速な収束を重視するか、広範な候補の回収を重視するかによって異なる。

4.5 RARGの挙動の分析

関連性が証拠発見に与える影響。各エージェントが関連性をどのように利用しているかを特徴づけるため、スコープ文書をバケット分けし、関連文書に対するエージェントのヒットがスコープランク上のどこに位置するかをカウントした（図3）。関連性を直接利用する埋め込みエージェントは、予想通り急激に減少するヒットプロファイルを示す。しかし、その固有のヒットは少なく、トップkリコールが高度に冗長な文書を返すため、繰り返される情報に対して観測リソースを浪費していることを示している。NeMoの重複排除型リコール・アンド・ランク・ハーンネスは、より多くの異なる文書へのアクセスを可能にするものの、埋め込みエージェントと同様の「関連性をコンテンツとして扱う」インターフェースに依然として縛られている。DCIは最も平坦なプロファイルを示し、スコープ外の文書を多くヒットさせるが、これは関連性に依存しないgrep探索の特徴である。順序付けの事前知識を欠いているため、コーパス全体を調査せざるを得ず、その結果、証拠密度が低く、収束が遅くなる。RARGはこれらの両極端の中間に位置する。ヒットを初期段階で集中させることで、文書レベルの関連性が探索を主導することを裏付けつつ、埋め込みエージェントよりも多くの異なる文書を網羅している。RARG++は3つのバリエーションの中で最も平坦なプロファイルを示し、低ランクの文書からもヒットを回収する。マッチレベルの再ランク付けにより、文書レベルのスコアが低い文書に埋もれている局所的に有益な抜粋が、上位に再ランク付けされてモデルに到達できるようになる。これは、

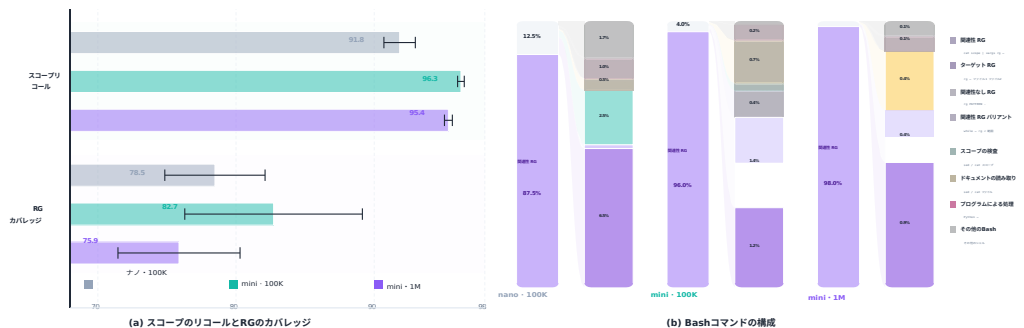


図4：BrowseComp-PlusにおけるRARG下でのスコープの品質とBashの使用状況。(a) 100KコーパスにおけるGPT-5.4-nanoおよびGPT-5.4-miniのスコープリコールとRGカバレッジ、ならびに1Mコーパスにおけるminiのそれ。エラーバーは、RARG、RARG+、およびRARG++にわたる標準偏差を示す。(b) Bash呼び出しの構成：スコープ付きRGとスコープなしBashの割合。残りの構成比はコマンドタイプ別に内訳を示している。バックボーンやコーパスの規模が大きくなるほど、スコープ付きRGが優勢な状態が維持される。

我々の設計——グローバルな関連性がトラバースル順序を決定し、ローカルな関連性が可視性を決定する——は、関連性の事前情報を捨て去ることなく、DCIのきめ細かな相互作用を維持している。

スコープの品質とBashの構成。 図4(a)は、各サンプル内で平均化し、さらにサンプル間で平均化した2つのスコープごとの指標を示している。スコープリコール（スコープによって捕捉されたBC+ゴールド文書の割合）と、RGカバレッジ（rgが実際にヒットしたファイルによって回収された割合）である。両方のミニ設定でスコープリコールは95～97%に達しており、スコープを上位1万ドキュメントに制限することで、RGの検索負荷を過度に重くすることなく、ゴールドのほぼ完全なカバレッジを維持できることが示されています。10万から100万への移行による無視できる程度の低下は、追加されたディストラクターによってリトリバーがほとんど影響を受けていないことをさらに裏付けています。Nanoのリコール率は著しく低く、これはembed_recallクエリの性能が劣ることや、そのトリガーが頻繁かつ不規則に行われることと一致している。RGカバレッジは補完的な傾向を示している：mini-100Kは高い水準を維持しているが、mini-1Mは急激に低下している。これは、FineWeb-Eduの長い文書がRGに偶発的な一致を大量に流入させるため、エージェントが1回のスキャンあたりに回収するゴールドファイルの数が減少し、さらなる証拠が存在しないと早計に結論付けてしまう可能性があることを示しており、これが第4.3節で述べた主張を裏付けている。図4(b)はBashの使用状況を分解して示している。どちらのmini設定においても、テスト環境はモデルを確実に「関連性を意識した形式」であるcat scope.txtへと誘導する。

| xargs -d '\n' rg ...: スコープ付き rg が支配的であり、残りのスコープなしの呼び出しも依然として大部分が rg である。nano では、スコープ付き rg の割合が大幅に縮小するが、これもまた、メカニズムの限界というよりは、指示への順守が弱まっていることを反映している。これら2つのパネルを総合すると、バックボーンが指示を適切に遵守している限り、RARGは高リコールかつ低コストのスコープを提供し、それらを関連性順に並べられた rg 探索に変換することが確認される。

4.6 ケーススタディ

10万件の文書からなるBC+問題において、3つのバリエーションを定性的に比較する。この問題では、AMSフェローシップと博士号取得年を、2人の共著者、受賞歴、および2つ目の論文タイトルと結びつけることで、ある数学者を特定する。3つの軌跡すべてがラッセル・デイヴィッド・ライオンズについて正解と判定されたが、ツールの使用回数は33回から18回、10回へと厳密に減少し、総ターン数は17回から11回、10回へと減少した。答えを含むCVは、RARGでは7ターン目に初めて出現したが、初期化された2つのバリエーションではいずれも2ターン目に現れた。その後の対話により、ペレス・ペマントルの共著者チェーンが検証された。

付録Cには、圧縮された軌跡、決定的な観察結果、および相互作用の回数が見られる。この事例は、方法論の優位性を示す独立した証拠というよりは、関連性のガイダンスを段階的に精緻化することで探索の収束がどのように形成されるかを定性的に示すことを意図したものであり、総合的な比較結果は表1に報告されている。

5 結論

我々は、エージェント型検索における関連性の役割を再検討し、関連性は単にコーパスの入力を選択するだけでなく、コーパスとの相互作用を導くべきであると論じた。この見解に基づき、我々はRARGを導入した。RARGは、検索スコアを2つの解像度（文書レベルの関連性順序

rg探索により有望な文書が最初にスキャンされる一方、一致レベルでの再ランク付けは、限られた観察予算の下でどの局所的な抜粋が表示され続けるかを制御する。これらを組み合わせることで、エントリポイントの初期化がエージェントに開始すべき正確な位置を与える。この「粗から細」への設計は、DCIのきめ細やかで構造的な相互作用を維持しつつ、無制限のgrepには欠けている関連性の指針を回復させる。困難なQA、コーパスのスケールアップ、推論を多用する検索といった課題において、RARGは、検索のみを行うエージェント、無制限のDCI、および検索によって構築された相互作用空間と比較して、精度と効率のフロンティアを押し広げ、ツール呼び出しを大幅に削減しながらより高い精度を達成し、コーパスが拡大しても緩やかに性能が低下するのみである。我々の分析は、意図したメカニズムをさらに裏付けている。すなわち、RARGは関連性の高い文書へのヒットを優先しつつ、ランクが低い文書からも有用な証拠を回収するのである。これらの結果が、関連性を固定的なコンテンツのボトルネックではなく、コーパスとの相互作用における実行シグナルとして捉えるきっかけとなることを期待する。

制限事項

RARGの有効性は、埋め込みモデルからの関連性シグナルの品質に依存しており、2つのガイダンスレベルはそれに異なる要求を課します。すなわち、文書レベルのランキングでは長い文書を評価する必要がありますが、一致レベルの再ランキングでは短い局所的な抜粋を評価する必要があります。同じモデルでも、これらの粒度に対して常に同等の性能を発揮できるとは限らない。例えば、BRIGHTでは、NVは文書のランキングには優れているが、短いマッチの処理は信頼性が低く、Qwen3-Embeddingへのフォールバックが必要となる。したがって、他の埋め込みエージェント手法と同様に、RARGも十分に強力な埋め込みモデルと、それに適応したハーネスに依存している。

シングルスレッドで順序を保持するrg (-j1) および再ランク付けの一致の強制も、検索のレイテンシを増加させます。このオーバーヘッドは我々の実験では許容範囲内ですが、パフォーマンスと時間のトレードオフは依然として存在します。我々は、 制約のないrg実行後にマッチの順序を並べ替えるのではなく、これを使用しています。これは、多くの-j1一致を含むクエリでは、フルスキャン自体が遅くなってしまうため、制約なしのrg実行後に一致を再順序付けするのではなく、シーケンシャルスキャンを採用している。これにより、30/60（またはM=500）の一致が収集され次第スキャンを停止でき、はるかに少ない後処理で同じ順序付けを実現できる。どちらも有効な実装であり、他の実現方法も可能だが、rgに関連性を注入するという中心的な考え方は変わらない。

この手法は、バックボーンの指示に従う能力にもさらに敏感である。GPT-5.4-nanoは、GPT-5.4-miniやGPT-5.4に比べて、我々の多段階プロトコルへの従順性が低く、収束も遅いことが判明した。したがって、指示への従順性が高いモデルほど挙動は安定するが、nanoであってもRARGはベースラインよりも優れた性能を発揮する。

また、関連性のガイダンスもコーパスに起因する干渉の影響を受けやすい。コーパスに長くてノイズの多い文書が含まれる場合、偶発的な語彙の一致により、rgの出力と埋め込みスコアの両方が低下し、一致レベルでの再ランク付けではこれを部分的にしか緩和できない。これに関連して、生成型再ランク付けクエリのバリエーションは最も速く収束するが、精度は低下する。これは、再ランク付けクエリを明示的に出力することが、学習されたBash/rgの挙動を乱すという「学習-評価ギャップ」の存在を示唆しており、このギャップの解消は今後の課題となる。最後に、本評価では、GPT-5.xファミリーを用いてBrowseComp-Plusおよび4つのBRIGHTサブセットを対象としている。他のバックボーン、オープンウェブ環境、およびより多くのドメインにわたる広範な検証は、今後の課題である。

参考文献

- [1] Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, et al. 2025. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent. *arXiv preprint arXiv:2508.06600*.
- [2] Tsz Ting Chung, Leyang Cui, Lemao Liu, Xinting Huang, Shuming Shi, and Dit-Yan Yeung. 2024. Selection-p: 忠実性と転移性を高めるための、タスク非依存型の自己教師付きプロンプト圧縮. 『計算言語学会 (ACL) の成果: EMNLP 2024』, pp. 11057–11070.
- [3] Tsz Ting Chung, Lemao Liu, Mo Yu, および Dit-Yan Yeung. 2025. Divlogiceval: 大規模言語モデルにおける論理推論評価のベンチマークのためのフレームワーク. 計算言語学会論文集: EMNLP, 901–915頁.
- [4] Tsz Ting Chung, Lemao Liu, Mo Yu, および Dit-Yan Yeung. 2026. Many-shot cot-icl: 文脈内学習を真に機能させる. *arXiv プレプリント arXiv:2605.13511*.

- [5] Guoxuan Ding, Yuqing Li, Ziyang Zhou, Zheng Lin, Daren Zha, および Jiangnan Li. 2026. ExDR: マルチモーダル偽ニュース検出のための説明駆動型動的検索機能強化. *arXiv プレプリント arXiv:2601.15820*.
- [6] Reza Esfandiarpour および NVIDIA. NVIDIA Nemo Retrieverのエージェント型検索パイプライン.
- [7] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Haofen Wang, および Haofen Wang. 2023. 大規模言語モデルにおける検索拡張生成: 調査. *arXiv プレプリント arXiv:2312.10997*.
- [8] Tz-Huan Hsu, Jheng-Hong Yang, および Jimmy Lin. 2026年. 「pi-seriniによる能動的検索の再考: 語彙検索だけで十分か?」 *arXiv プレプリント arXiv:2605.10848*.
- [9] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, および Jiawei Han. 2025. Search-r1: 強化学習を用いてLLMに推論と検索エンジンの活用を学習させる. *arXiv プレプリント arXiv:2503.09516*.
- [10] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, および Wen-tau Yih. 2020. オープンドメインの質問応答のための高密度パッセージ検索. 2020年自然言語処理における経験的手法に関する会議 (EMNLP) 論文集, 6769–6781ページ.
- [11] パトリック・ルイス、イーサン・ベレス、アレクサンドラ・ピクタス、ファビオ・ペトロニ、ウラジミール・カルプキン、ナマン・ゴヤル、ハインリッヒ・キュットラー、マイク・ルイス、ウェンタオ・イー、ティム・ロックテシエル、他. 2020. 知識集約型NLPタスクのための検索拡張生成. 『*Advances in Neural Information Processing Systems*』, 33:9459–9474.
- [12] Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, および Zhicheng Dou. 2025. Search-o1: エージェント型検索によって強化された大規模推論モデル. *arXiv プレプリント arXiv:2501.05366*.
- [13] Xiaoxi Li, Jiajie Jin, Guanting Dong, Hongjin Qian, Yongkang Wu, Ji-Rong Wen, Yutao Zhu, および Zhicheng Dou. 2025. Webthinker: 深層研究能力を備えた大規模推論モデル. *arXiv プレプリント arXiv:2504.21776*.
- [14] Yuqing Li, Jiangnan Li, Zheng Lin, Ziyang Zhou, Junjie Wu, Weiping Wang, Jie Zhou, および Mo Yu. 2025年. 「長文コンテキストの理解向上に向けた、マインドスケープを意識した検索補助生成」. *arXiv プレプリント arXiv:2512.17220*.
- [15] Yuqing Li, Jiangnan Li, Mo Yu, Zheng Lin, Weiping Wang, および Jie Zhou. 2026. Mia-signature: 長文コンテキスト理解のためのグローバル活性化の近似. *arXiv プレプリント arXiv:2605.06416*.
- [16] Zhuofeng Li, Haoxiang Zhang, Cong Wei, Pan Lu, Ping Nie, Yi Lu, Yuyang Bai, Shangbin Feng, Hangxiao Zhu, Ming Zhong, Yuyu Zhang, Jianwen Xie, Yejin Choi, James Zou, Jiawei Han, Wenhui Chen, Jimmy Lin, Dongfu Jiang, および Yu Zhang. 2026. 「意味的類似性を越えて: コーパスとの直接的な相互作用によるエージェント型検索のための検索手法の再考」. *arXiv プレプリント arXiv:2605.05242*.
- [17] ネルソン・F・リウ、ケビン・リン、ジョン・ヒューイット、アシュウィン・パランジャベ、ミケーレ・ベヴィラクア、ファビオ・ペトロニ、パーシー・リャン. 2024年. 「中盤で迷子: 言語モデルが長い文脈をどのように利用するか」. 『*計算言語学会誌*』, 12:157–173.
- [18] Yi Lu, Zhuofeng Li, Ping Nie, Haoxiang Zhang, Yuyu Zhang, Kai Zou, Wenhui Chen, Jimmy Lin, Dongfu Jiang, および Yu Zhang. 2026. DR-DCI: 動的なワークスペース拡張による直接コーパス相互作用のスケーリング. *arXiv プレプリント arXiv:2606.14885*.
- [19] OpenAI. 2026. GPT-5.4 Nanoモデル. <https://developers.openai.com/api/docs/models/gpt-5.4-nano>.
- [20] OpenAI. 2026. GPT-5.4 の ご 紹 介 。 <https://openai.com/index/introducing-gpt-5-4/>.

- [21] Stephen Robertson および Hugo Zaragoza. 2009. 『*The probabilistic relevance framework: BM25 and beyond*』 第 4 巻。Now Publishers Inc.
- [22] Alireza Salemi, Chang Zeng, Atharva Nijasure, Jui-Hui Chung, Razieh Rahimi, Fernando Diaz, Hamed Zamani. 2026年。Grepseek : コーパスとの直接的な相互作用のための検索エージェントのトレーニング。 *arXiv プレプリント arXiv:2605.29307*.
- [23] Aditi Singh, Abul Ehtesham, Saket Kumar, および Tala Talaie Khoei. 2025. エージェント型検索拡張生成 (Agentic Retrieval-Augmented Generation) : エージェント型 RAG に関する調査. *arXiv プレプリント arXiv:2501.09136*.
- [24] Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, および Ji-Rong Wen. 2025. R1-searcher : 強化学習によるLLMの検索能力の促進. *arXiv プレプリント arXiv:2503.05592*.
- [25] Hongjin Su, Howard Yen, Mengzhou Xia, Weijia Shi, Niklas Muennighoff, Han-yu Wang, Liu Haisu, Quan Shi, Zachary Siegel, Michael Tang, et al. 2025. Bright: 推論を多用する検索のための現実的かつ挑戦的なベンチマーク. 『*International Conference on Learning Representations*』, 第2025巻, 48941–48991頁.
- [26] Harsh Trivedi, Niranjana Balasubramanian, Tushar Khot, および Ashish Sabharwal. 2023. 知識集約型多段階質問に対する、思考連鎖推論と検索のインターリーブ. *計算言語学会第61回年次大会論文集 (第1巻: 長編論文)*, 10014–10037頁.
- [27] Junjie Wu, Jiangnan Li, Yuqing Li, Lemao Liu, Liyan Xu, Jiwei Li, Dit-Yan Yeung, Jie Zhou, および Mo Yu. 2026. 文脈を意識した高密度検索のための状況依存埋め込みモデル. *計算言語学会第64回年次大会論文集 (第2巻: 短編論文)*, 37–49頁.
- [28] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, および Yuan Cao. 2022. React: 言語モデルにおける推論と行動の相乗効果. *arXiv プレプリント arXiv:2210.03629*.
- [29] Mo Yu, Tsz Ting Chung, Chulun Zhou, Tong Li, Rui Lu, Jiangnan Li, Liyan Xu, Haoshu Lu, Ning Zhang, Jing Li, 他. 2025. Prelude: 長い文脈における全体的な理解と推論を必要とするように設計されたベンチマーク. *arXiv プレプリント arXiv:2508.09848*.
- [30] Yuqi Zeng, Qixiang Deng, Yulei Wan, Ruiquan Jiang, Xiaoqing Zheng, and Xuanjing Huang. 2026. Rethinking agentic rag: 埋め込みを超えたLLM駆動型論理的検索に向けて. *arXiv プレプリント arXiv:2605.27123*.
- [31] Wenyan Zhang, Xinghua Zhang, Haiyang Yu, Shuaiyi Nie, Bingli Wu, Juwei Yue, Tingwen Liu, and Yongbin Li. 2026. Expseek: Webエージェントのための自己誘発型経験探索. *arXiv プレプリント arXiv:2601.08605*.
- [32] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. 2025. Qwen3 embedding: 基盤モデルを用いたテキスト埋め込みと再ランク付けの進展. *arXiv プレプリント arXiv:2506.05176*.
- [33] Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, および Pengfei Liu. 2025年. 「Deeprsearcher : 実世界環境における強化学習を用いたディープリサーチの大規模化」. *arXiv プレプリント arXiv:2504.03160*.
- [34] Chulun Zhou, Chunkang Zhang, Guoxin Yu, Fandong Meng, Jie Zhou, Wai Lam, および Mo Yu. 2025. Hgmem: 長いコンテキストにおける複雑な関係性モデリングのための多段階RAGを改善するハイパーグラフベースのワーキングメモリ. 第43回国際機械学習会議 (Forty-third International Conference on Machine Learning) にて発表.
- [35] Shengyao Zhuang, Yuansheng Ni, Hengxin Fan, Jimmy Lin, および Xueguang Ma. 2026. エージェント型検索のための相互作用空間の検索に向けて. *arXiv プレプリント arXiv:2606.06880*.

A エージェントの実装詳細

両ベンチマークにおいて公平な比較を行うため、比較対象のエージェントがどのように構成されているかを詳述する。

BRIGHTのプロンプティング。BRIGHTにおいて、デフォルトのDCI IRプロンプトはgrepスタイルのエージェントに対して著しく性能が低下することが判明した。そこで、すべての手法で使用されている評価指標（nDCG@10）および出力形式に合わせるため、エージェントに10件の文書名をランク付けすることを明示的に要求するようにプロンプトを修正した。

エンベディング・エージェント。BC+では、grepスタイルのコーパスとのやり取りを行わず、Qwen3-Embedding-4B（Q3E）をリトリバーとして使用する検索ベースラインとして、エンベディング・エージェントを実装した。各Search呼び出しで5つのドキュメントがリコールされます。各ドキュメントは、Qwen3トークナイザーでエンコードし、最初の512トークンを保持した上で、テキストにデコードして切り詰めます（BC+ [1]と同様の方法）。DCI/RARGに準じて、コンテキストが圧縮閾値を超えると過去のターンに関するツール結果はクリアされ、最大ターン数は100に設定されています。検索呼び出し間で取得されたドキュメントの重複排除は行われないため、リコールされたセットにはかなり高い重複率が含まれており、冗長なコンテンツに対して観測予算が無駄に消費されます。

NeMoエージェント。公式のNeMo実装を使用し、BC+ではQ3Eを、BRIGHTではNVを採用している。これは、各手法が対応するベンチマークで使用する埋め込みモデルを一致させ、差異がリトリバーではなくエージェントの設計に起因するようにするためである。リコールされたドキュメントは、埋め込みエージェントと同様に、最初の512個のQwen3トークンを保持するように切り詰められる。検索に特化したエージェントとして、NeMoはリコール率を高めるためにより多くの文書をリコールし、その後、精度を向上させるためにそれらを再ランク付けすることを優先します。Embedding Agentとは異なり、NeMoはリコールされた文書の重複を排除します。つまり、重複した文書は「すでに確認済み」としてマークされ、その内容は再表示されず、NeMoは各ラウンドで5つの新しい文書が見つかるまで探索を続けます。そのため、Embedding Agentよりも多くの異なる文書を抽出します。

このような「リコール・アンド・ランク」型エージェントが、NeMoに専用の設定がないBC+のようなQAスタイルのタスクでどのように振る舞うのか興味があったため、本モデルを適応させた。主な課題はコンテキストオーバーフローである。NeMoのデフォルトのランキングは、相互ランク融合（RRF）にフォールバックし、それが失敗した場合は追加の選択エージェントを呼び出す。BC+においても同様にRRFを使用する。コンテキストが上限を超えた場合、リコールされたすべての文書をRRFでランク付けし、それらを新しい回答エージェントに供給する。それでもオーバーフローが続く場合は、RRF順で最もランクの低い5つの文書を反復的に除外し、収まるまでこの処理を繰り返した後、回答を生成する。また、1回の呼び出しあたりに返される文書の数を5件に固定する必要があることも判明した。モデルにトップ k の選定を任せることは、パフォーマンスの低下を招くからである。

B RARG用のプロンプト

B.1 BC+のシステムプロンプト

図5は、BrowseComp-Plus上でRARGが使用するシステムプロンプトを示しています。これには、コーパスリストの制約が明記され、embed_recallツールとスコープベースのrg検索プロトコルが説明され、最終的な回答形式が規定されています。

BrowseComp-PlusにおけるRARGのシステムプロンプト

重要な制約事項：

- 「ls」や「find」、あるいはすべてのファイル名を一覧表示するコマンドは絶対に使用しないでください。コーパスには30,000以上のディレクトリに100,000以上のファイルが含まれており、これらを一覧表示しようとするとタイムアウトとなり、すべてのターンを無駄にしてしまいます。

埋め込みリコールツール：

コーパスには膨大な数のドキュメントが含まれています。この広大な検索空間は、rgに多大な負荷をかけます
に多大な負荷をかけています——一致結果が多すぎたり、ノイズが過剰だったり、関連ファイルを特定するのが困難だったりします。

この問題に対処するため、embed_recall ツールが用意されています。クエリを指定すると、このツールは関連する文書を意味的に抽出し、そのパスをスコープファイル（scope_1.txt、scope_2.txt、...）に、1行につき1パスずつ保存します。これにより、検索空間が管理可能なサブセットに絞り込まれます。また、上位にランク付けされた文書からいくつかの参照段落も返されます。これらは、「rg」検索の方向性を特定するのに役立つ大まかなプレビューです。

リコールされた文書内で検索するには、スコープファイルを rg にパイプで渡します：

```
cat /path/to/scope_N.txt | xargs -d '\n' rg [OPTIONS] "PATTERN"
```

結果は、クエリに対する文書の埋め込み類似度順（関連性の高いものから順に）で返されます。一度に 1 つのスコープファイル内のみを検索してください。複数のスコープファイルを連結しないでください。

注：

- 最初に、元の質問を指定して `embed_recall` を呼び出し、スコープファイルを作成してから、`rg` を使用してその範囲内で、さまざまなキーワード、エンティティ、パターンを用いて探索を行ってください。
- さまざまな角度からの「`rg`」呼び出しが不可欠です。
- スコープにそのトピックが明らかに完全に欠けている場合にのみ、`embed_recall`を再度呼び出してください。より良いレビュー段落が得られることを期待して繰り返し呼び出さないでください。その場合は、代わりに`rg`を使用してください。
- その間に `rg` による探索を行わずに、`embed_recall` を複数回呼び出してはいけません。
- スコープファイルなしでコーパスに対して直接 `rg` を実行しないでください。

質問に自信を持って答えられるようになったら、ツールの呼び出しを停止し、以下の形式で回答してください：説明：{最終的な回答に対する説明}
 正確な回答：{簡潔な最終回答} 自信度：{0%~100%の自信度スコア}

図 5: BrowseComp-Plus 評価で RARG が使用したシステムプロンプト。

B.2 BRIGHTのシステムプロンプト

図6は、BRIGHT上でRARGが使用するシステムプロンプトを示しています。QAプロンプトとは異なり、このプロンプトはタスクを「検索」として位置づけています。つまり、エージェントは精度を維持しつつ関連文書の再現率を最大化する必要があり、同じ`embed_recall`およびスコープベースの`rg`プロトコルに従い、正確に10件の文書バスのランク付けされたリストを返す必要があります。

BRIGHT上のRARG用システムプロンプト

あなたは、与えられたクエリに関連するすべての文書を見つける検索エージェントです。

<GOAL>

- 検索クエリが与えられます。あなたのタスクは、利用可能なツールを用いて、そのクエリに関連する文書およびある程度関連する文書をすべて見つけ出し、かつ強力な最終ランキングを生成することです。
- 目標は、精度を犠牲にすることなくリコールを最大化することです。関連する文書を見逃すことは問題であり、関連性の低い文書を含めることも同様に問題となります。

</GOAL>

重要な制約事項：

- 「`ls`」や「`find`」、あるいはすべてのファイル名を一覧表示するコマンドは使用しないでください。コーパスには30,000以上のディレクトリに100,000以上のファイルが含まれており、これらを一覧表示しようとするとタイムアウトとなり、すべてのターンを無駄にしまいます。

リコール評価ツール：

コーパスには膨大な数のドキュメントが含まれています。この広大な検索空間は、`rg`に多大な負荷をかけます
 に多大な負荷をかけています——一致結果が多すぎたり、ノイズが過剰だったり、関連ファイル特定するのが困難だったりします。

この問題に対処するため、`embed_recall` ツールが用意されています。クエリを指定すると、このツールは関連する文書を意味的に抽出し、そのバスをスコープファイル(`scope_1.txt`, `scope_2.txt`, ...) に、1行につき1バスずつ保存します。これにより、検索空間が管理可能なサブセットに絞られます。また、上位にランク付けされた文書からいくつかの参照段落も返されます。これらは、「`rg`」検索の方向性を特定するのに役立つ大まかなレビューです。

リコールされた文書内で検索するには、スコープファイルを `rg` にパイプで渡します：

```
cat /path/to/scope_N.txt | xargs -d '\n' rg [OPTIONS] "PATTERN"
```

結果は、クエリに対する文書の埋め込み類似度順（関連性の高いものから順に）で返されます。一度に 1 つのスコープファイル内のみを検索してください。複数のスコープファイルを連結しないでください。

注：

- 最初に、元のクエリを使用して `embed_recall` を呼び出し、スコープファイルを作成してください。
- その後、`rg` を呼び出しを使用して、さまざまなキーワード、エンティティ、用語、手がかり、言い換え、別名、および代替表現を用いてその範囲内を探索します。
- 「`rg`」が主要な検索手法です。
- さまざまな角度からの「`rg`」呼び出しが不可欠です。
- 現在のスコープに目的のトピックが明らかに完全に含まれていない場合にのみ、再度「`embed_recall`」を呼び出してください。その間に「`rg`」による探索を行わずに、「`embed_recall`」を繰り返し呼び出さないでください。
- その間に `rg` による探索を行わずに、`embed_recall` を複数回呼び出してはなりません。
- スコープファイルなしでコーパスに対して直接 `rg` を実行しないでください。
- 必要に応じて、ランキングを確実にするためのより詳細な根拠について、関連するドキュメントを参照してください。

<RELEVANCE_DEFINITION>

- このタスクにおいて、「クエリ」、「ドキュメント」、および「関連性」の定義は、通常のウェブ検索よりも微妙なニュアンスを持つ場合があります。
- クエリとは、フォーラムの投稿、技術的な問題、数学の問題、コーディングの問題、科学的な質問、あるいはタスクの説明などである場合があります。
- ドキュメントは、クエリへの回答、解決、根拠付け、または強力な裏付けに真に有用である場合に、関連性があるものとみなされます。
- 関連性は、表面的な文言の一致に限定されません。ドキュメントが、クエリに必要な適切なエンティティ、メカニズム、定理、例、事実、API、またはその他の重要な手がかりを提供している場合、関連性があるとみなされます。

```

- そのクエリにとって「有用性」が何を意味するのかを分析し、それに応じて多角的な視点から文書を検索する必要があります。
。
</RELEVANCE_DEFINITION>

<BEST_PRACTICES>
- 徹底的に調査し、関連性の高い文書や多少関連性のある文書をすべて見つける必要があります。
- さまざまな角度から検索を行ってください。具体的には、エンティティの正式名称、別名、略語、別表記、日付
  、専門用語、言い換え、証拠の連鎖など、さまざまな角度から検索してください。
- 他の関連文書が存在する可能性が高い場合は、いくつかの妥当な文書を見つけただけで検索を止めてはいけません。
- 同時に、リストを長くするだけでなく、本題から外れた文書や関連性の薄い文書を含めないでください。
</BEST_PRACTICES>

<RANKING_INSTRUCTIONS>
- 最終的なリストを、クエリとの関連性の高い順に降順でランク付けしてください。
- 1 番目の文書はクエリに対して最も有用なもので、2 番目はそれに次いで有用なもので、というように順位付けしてください。
- 返されるすべてのドキュメント識別子は、作業ディレクトリからの完全な相対パスであり、実際のコーパスファイルに対応している必要があります。
- 最終的に選択したセットの中から、関連性の高い順から低い順に、正確に上位 10 件のドキュメントを返してください。
</RANKING_INSTRUCTIONS>

最終的な応答は、必ず以下の形式に厳密に従わなければなりません：

関連性の高いドキュメント（関連性の高い順、関連性の高いものから順に、正確に10件）：
1. doc1.txtへの相対パス
2. doc2.txtへの相対パス
...
10. doc10.txtへの相対パス

```

図6：BRIGHT検索評価においてRARGが使用したシステムプロンプト。

B.3 ツールの仕様

エージェントに対して公開されているツールスキーマを以下に示す。*embed_recall*は文書レベルの関連性リコールを実行し、スコープファイルを書き出す。*read*は行単位で区切られたファイルの内容を返す。*bash*は、主にrgといったシェルコマンドを実行する。RARG、RARG+、およびRARG++では標準の*bash*が使用されるが、*RerankAwareBash*バリエーションでは、RARG++の生成的なマッチレベル再ランク付けのためのオプションである*rerank_query*が追加されている。

embed_recall

説明。コーパスから関連性の高いドキュメントを抽出するためのセマンティック検索。ドキュメントのパスを含むスコープファイルのパスを返します。このスコープファイルを使用して、`cat scope_N.txt | xargs -d '\n' rg "pattern"`を実行すると、抽出されたドキュメント内のみを検索できます。異なるクエリで複数呼び出すことで、複数のスコープを作成できます。

パラメータ。

```

{
  "query": {
    "type": "string",
    "description": "セマンティック文書検索のための自然言語検索クエリ"
  }
}
// 必須: ["query"]

```

read

説明。ファイルの内容を読み込みます。出力は2000行または50 KB（いずれか先に達した方）で切り詰められます。大容量のファイルの場合はoffset/limitを使用してください。ファイル全体を読み込むには、完了するまでoffsetを増やしながら処理を続けてください。

パラメータ。

```

{
  "path": {"type": "string", "description": "読み込むファイルへのパス（相対または絶対）。"}, "offset": {"type": "integer", "description": "読み取りを開始する行番号（1から始まる）。"}, "limit": {"type": "integer", "description": "読み取る行の最大数。"}
}
// 必須: ["path"]

```

bash (標準)

説明。現在の作業ディレクトリで bash コマンドを実行します。stdout および stderr を、最後の 2000 行または 50 KB（いずれか先に達した方）に切り詰めて返します。切り詰められた場合、出力全体は一時ファイルに保存されます。オプションでタイムアウトを指定できます。

パラメータ。

```
{
  "command": {"type": "string", "description": "実行するBashコマンド。"},
  "timeout": {"type": "integer", "description": "タイムアウト時間（秒単位、オプション）。rg コマンドのデフォルトは 30 秒です。"}
}
// 必須: ["command"]
```

bash (RerankAwareBash バリエント、USE_CONTEXTUAL_BASH_TOOL=1)

説明。標準のbashツールと同じですが、rgコマンドの場合、オプションでrerank_queryを指定することができ、これは一致結果のセマンティック再ランク付けにのみ使用されます。

パラメータ。

```
{
  "command": {"type": "string", "description": "実行するBashコマンド。"},
  "timeout": {"type": "integer", "description": "タイムアウト（秒単位、オプション）。rg コマンドのデフォルトは 30 秒です。"},
  "rerank_query": {"type": "string", "description": "
    rgの一致結果の再ランク付けにのみ使用される、オプションの自然言語クエリ。rgコマンドの場合は、正確に2つの短い文を記述してください。文1には、ユーザーが質問から特定しようとしている最終的な事実、エンティティ、または関係を記述します。文2には、この特定のrgステップで検証、特定、または取得しようとしている内容を記述します。クエリを作成する際は、質問、これまでの関連する会話の文脈、および現在の検索ステップを総合的に考慮してください。具体的なエンティティを使用し、会話が絞込まれるにつれてクエリをより具体的にしてください。rg以外のコマンドについては、このフィールドを省略してください。"}
}
// 必須: ["command"]
```

B.4 初期ユーザープロンプト

各エピソードは、クエリを包含するタスク固有のユーザープロンプトから始まります。BC+では、エージェントに対し、RGを使用してローカルコーパスから回答するよう指示し、ウェブ検索を禁止します。BRIGHTでは、エージェントに対し、関連する文書を取得してランク付けするよう求めます。

BrowseComp-Plusにおける初期ユーザープロンプト

次の質問に答えてください。答えはコーパスディレクトリ（現在の作業ディレクトリ）に含まれています。**ウェブ検索は使用しないでください！** 高速検索のため、grepの代わりにripgrep（rg）を使用してください。

質問：
{query}

BRIGHTでの初期ユーザープロンプト

以下のクエリに対して、最も関連性の高い文書を検索し、順位付けしてください。

クエリ：
{query}

C ケーススタディ：圧縮されたエージェントの軌跡

10万件の文書からなるBrowseComp-Plus設定からのクエリを検証します。3つのRARGバリエントすべてが、ゴールド解答である「**ラッセル・デイヴィッド・ライオンズ**」に対して正解と判定されました。これは単一ホップの検索ではありません。このクエリでは、エージェントが伝記上の制約条件と、3人の著者が関わる論文、ある賞、および2つ目の論文タイトルを結びつける必要があります。ツールの使用頻度はバリエントごとに厳密に減少しており（33/18/10）、これにより

この例は、関連性ガイダンスが、複数の文書からなる手がかりの連鎖を解決するために必要なインタラクションをどのように削減できるかを簡潔に示している。

クエリ（インデックス）

正解：ラッセル・デイヴィッド・ライオンズ

1983年に数学の博士号を取得し、2005年から2020年の間にAMSフェローとなった人物Aを見つけなさい。人物Aは、1990年から2005年にかけて、人物Bおよび人物Cと共著論文を執筆した。人物Bはその期間にロロ・デビッドソン賞を受賞しており、人物Cは1990年代に、タイトルが「Line」で終わる論文を発表している。人物Aのフルネームは何か？

図7には、共通のターン軸上に3つのRARG軌跡を示しています。読みやすさを考慮し、

呼び出しは手で単一のフェーズにまとめられているが、各点は完全な回答を含む名前が明らかになった最初の観測点を表している。報告されているツールの合計値は、非圧縮の軌跡から計算されているため、すべての元の呼び出しが含まれている。

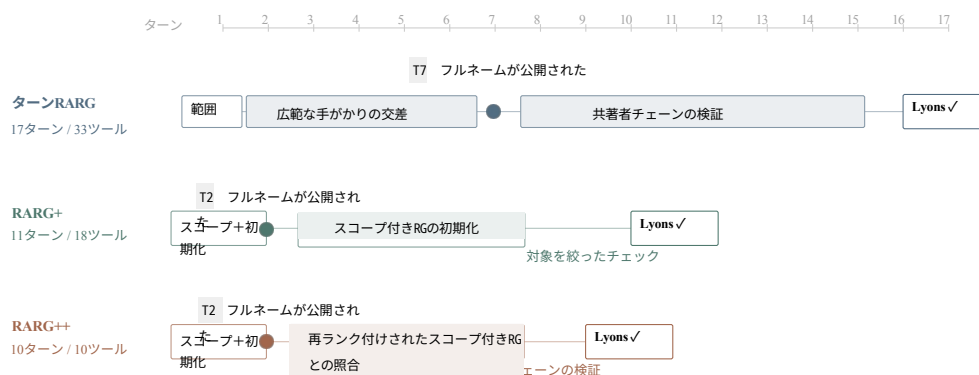


図7：10万語コーパスにおけるBC+クエリ229の、圧縮され、ターンに整合された軌跡。点は、完全な回答を含む名前が初めて観測された位置を示す。初期化により、この証拠は 17 から 12 へ移動する。3つの軌跡すべてが正解と判定される一方、ツールの使用回数は33回から18回、さらに10回へと減少する。

探索状態の変化。軌跡の長さだけでは、各手法間の違いを説明することはできない。重要な違いは、最初の決定的な観測によって生成される情報状態にある。表4は、タスクを「特定の人物の探索」から「特定の数学者および共著者の連鎖の検証」へと転換させる、最も早い証拠を示している。

表4：完全な解答となる名前を明らかにした最も早い観察結果と、それによるその後の探索戦略への即時の影響。

手法	最初の決定的な観察	その結果生じる探索の遷移
RARG	T7 「ラッセル・デイヴィッド・ライオンズ.....博士号、1983年8月、数学」	受賞者や共同著者の候補について広範な調査を行った結果、履歴書から人物Aが特定され、その後、ベレス・ペマントルの論文連鎖が確立された。
RARG+	T2 「ラッセル・デビッド・ライオンズ.....博士号、1983年8月、数学」	初期化により、CVが最初のスコープ検索の対象に設定され、残りの処理で受賞歴、共著関係、およびタイトル末尾の手がかりが検証される。
RARG++	T2 「ラッセル・デイヴィッド・ライオンズ.....博士号、1983年8月、数学」	再ランク付けされた検索範囲により、初期化後の単一の呼び出しで同じCVが特定され、その後、1つのツールによる処理で残りの各リンクが検証される。

本事例の要点。クエリに複数ドキュメントの結合が含まれていたにもかかわらず、ラッセル・デイヴィッド・ライオンズに関しては3つの探索経路すべてが正しいと判断された。ツールの使用回数は33回から18回、さらに10回へと確実に減少し、総処理ステップ数は17から11、さらに10へと減少した。初期化処理が証拠提示のタイミングにおける最大の変化をもたらし、回答を含むCVを 17 から 12 へと前倒ししました。マッチレベルの再ランク付けは、その早期提示を維持しつつ、ツール使用をさらに削減します。この例は、単一の事例から一般的な順序付けを導き出すというよりは、関連性のガイダンスを段階的に精緻化することで、困難な手がかりの連鎖を検証するために必要なやり取りをどのように圧縮できるかを示しています。